

KI PHPUnit Test Generator

Erläuterung der drei Dozenten-Empfehlungen zur Projektskizze

Der Dozent hat drei Empfehlungen zur eingereichten Projektskizze gegeben. Dieses Dokument erklärt jeden Punkt ausführlich und zeigt auf, was konkret zu tun ist.

Punkt 1 – Szenarien priorisieren

Der Dozent möchte, dass du nicht allgemein von «TYPO3 Backend-Entwicklung» sprichst, sondern konkret festlegst, welche Art von PHP-Code du testen willst. TYPO3-Projekte sind sehr verschieden – ein Custom Module hat andere Test-Anforderungen als ein komplexes Datenmodell oder eine Systemintegration.

Was sollst du konkret machen?

Wähle 1–2 klar abgegrenzte Szenarien und begründe deine Wahl:

Custom Module

Backend-Plugin mit Service-Klassen. Gut geeignet, da die Logik isoliert und ohne Framework-Abhängigkeit testbar ist.

Komplexes Datenmodell

Repository-Klassen mit TCA-Mapping, Extbase-Entities. Interessant für Mocking von TYPO3-Abhängigkeiten.

Systemintegration

API-Clients, DataHandler-Aufrufe. Anspruchsvoller, da viele Mocks benötigt werden.

Workflow-Automatisierung

Scheduler-Tasks, Event-Handler. Gut für zeitorientierte Logik und Zustandstests.

Empfehlung: Nutze dein eigenes biber-Projekt als Basis. Wähle 3–5 reale Klassen daraus als Testfälle. Das gibt dir sofort einen konkreten, messbaren Umfang und erleichtert den späteren Vergleich (manuell vs. KI-generiert).

Punkt 2 – Übersichtszeichnung

Deine Projektskizze beschreibt WAS du machen willst, aber nicht WIE alles zusammenhängt. Eine Übersichtszeichnung zwingt dich, den vollständigen Ablauf zu durchdenken, bevor du anfängst zu programmieren.

Warum will der Dozent das?

- Wo beginnt der Prozess, wo endet er?
- Was sind die Ein- und Ausgaben jedes Schritts?
- Wie misst du am Ende, ob dein Tool gut ist?

Der Dozent will sehen, dass du das Gesamtbild verstehst – nicht nur den KI-Teil in der Mitte.

Abschnitt 1: «Vom Generieren der Tests» – Input & Vorbereitung

Dieser Teil zeigt, wie der PHP-Code ins System kommt und was passiert, bevor der LLM ihn erhält:

```

PHP-Klasse (z.B. DateCalculator.php)
|
Code-Analyse
- Welche public Methoden gibt es?
- Welche Abhängigkeiten? (DI, Repos, Services...)
- Gibt es bereits DocBlocks / Kommentare?
|
Prompt wird zusammengebaut
- Systemanweisung: "Du bist TYPO3-Experte"
- PHP-Code wird eingefügt
- Anweisung: "Generiere PHPUnit-Tests mit DataProvider"
|
LLM (Claude API)
|
Ausgabe: PHP-Testklasse als Text

```

Abschnitt 2: «Zur Nutzung und Effekt» – Ausführung & Validierung

Dieser Teil zeigt, was NACH der Generierung passiert – der Dozent will explizit sehen, dass du diesen Schritt mitdenkst:

```

Generierter Test wird gespeichert
(z.B. Tests/Unit/DateCalculatorTest.php)
|
PHPUnit wird ausgeführt
(via DDEV: ddev exec phpunit ...)
|
Ergebnis:
OK --> Test läuft durch --> sofort nutzbar
FAIL --> Fehler --> manuell nachbessern
|
Effekt messen:
- Code Coverage vorher / nachher (%)
- Wie viele Tests laufen ohne Änderung durch?
- Zeitaufwand: Generierung + Korrektur (Minuten)

```

Abschnitt 3: «Bis zum Vergleich» – Bewertung & Erkenntnisse

Hier liegt der wissenschaftliche Wert der Arbeit. Dieselbe PHP-Klasse wird auf zwei Wegen getestet:

Weg A: Manuell	Weg B: KI-generiert
- Entwickler schreibt Tests	- Tool generiert Tests
- Zeitaufwand: X Minuten	- Zeitaufwand: Y Min. (inkl. Korrektur)
- Qualität: subjektiv	- Qualität: objektiv messbar

Vergleichskriterien:

Laufen Tests durch?	(ja / nein / nach Korrektur)
Code Coverage	(%)
Sinnvolle Assertions?	(manuelle Bewertung 1-5)
Edge-Cases abgedeckt?	(ja / teilweise / nein)
Zeitersparnis?	(Minuten gespart pro Klasse)

Kernbotschaft: Ohne Vergleich ist dein Projekt nur ein Tool, das Tests generiert. MIT Vergleich beantwortest du die eigentliche Forschungsfrage: «Sind KI-generierte Tests gut genug, um manuell erstellte zu ersetzen oder zu ergänzen?» – Das ist der wissenschaftliche Kern, den der Dozent sehen will.

Empfohlene Tools für die Zeichnung

- Handskizze mit Foto – reicht für die Projektskizze
- Draw.io (draw.io) – kostenlos, im Browser, Export als PNG/PDF
- Mermaid – Diagramm als Markdown-Code, direkt in Dokumentation einbettbar

Punkt 3 – Rolle des Anforderungswissens

Das ist die tiefste und wissenschaftlich interessanteste Frage des Dozenten. Er weist darauf hin, dass Code alleine nicht ausreicht, um GUTE Tests zu generieren.

Was ist damit gemeint?

Ein LLM sieht nur den Quellcode. Aber Code sagt nicht, was das System TUN SOLL – nur, was es aktuell tut. Beispiel:

```
public function calculateDiscount(int $amount): int
{
    return $amount * 0.1;
}
```

Ein LLM kann daraus generieren: `assertEquals(10, $obj->calculateDiscount(100))`

Aber folgende Fragen bleiben offen:

- Soll der Rabatt nie negativ sein?
- Gibt es eine maximale Rabattgrenze laut Geschäftsregel?
- Was passiert bei Eingabe 0 oder negativen Werten?
- Gibt es eine Währungsroundingslogik?

Diese Regeln stehen NICHT im Code – sie stehen in Tickets, Specs, Kommentaren oder nur im Kopf des Kunden. Das ist «Anforderungswissen».

Was sollst du konkret machen?

Füge diese Kernfrage explizit in dein Projekt ein:

Neue Kernfrage: Welchen Mehrwert bringt es, dem LLM zusätzlich zum Code auch Anforderungen (z.B. Ticket-Beschreibungen, PHPDoc, Kommentare) als Kontext zu geben?

Plane einen strukturierten Vergleich in zwei Varianten:

Variante A – Code only

LLM erhält nur den PHP-Quellcode.

Ergebnis: Technisch korrekte, aber oberflächliche Tests.

Fehlende Edge-Cases, keine Geschäftsregel-Assertions.

Variante B – Code + Kontext

LLM erhält Code + PHPDoc + Ticket-Beschreibung / Kommentare.

Ergebnis: Tests prüfen auch Geschäftsregeln, Grenzwerte,

Fehlerfälle die im Code nicht direkt sichtbar sind.

Warum ist das wissenschaftlich relevant?

Der Vergleich Variante A vs. B zeigt, ob und wie viel Anforderungswissen die Test-Qualität verbessert. Das führt zu einer der wertvollsten Erkenntnisse der Arbeit:

- Für welche Klassen reicht Code alleine?
- Wann ist zusätzlicher Kontext zwingend notwendig?

- Welche Form von Kontext hilft am meisten? (PHPDoc, Kommentar, Ticket-Text)

Zusammenfassung Punkt 3: Der Dozent will, dass du anerkennst, dass Code-only-Analyse Grenzen hat. Anforderungswissen kann die KI-Ausgabe erheblich verbessern – und ob das zutrifft, ist eine messbare, beantwortbare Forschungsfrage.

Zusammenfassung aller drei Punkte

Punkt 1 – Scope eingrenzen

Konkrete Szenarien wählen (z.B. Custom Module + Datenmodell). Reale Klassen aus dem eigenen Projekt als Testbasis definieren.

Punkt 2 – Ablauf visualisieren

Flussdiagramm erstellen: Input Analyse LLM PHPUnit ausführen Ergebnis messen Vergleich manuell vs. KI.

Punkt 3 – Anforderungswissen einbeziehen

Forschungsfrage ergänzen: Was ändert sich, wenn dem LLM nicht nur Code, sondern auch Anforderungen (Tickets, DocBlocks) gegeben werden?